

JUDGEAI ? TESTING & FEATURE HANDBOOK

PURPOSE

This document provides a complete, step-by-step guide to test the JudgeAI application from scratch, verify key features, and understand architecture, AI/ML model usage, and future enhancement scope.

SECTION 1 ? PRE-TEST CHECKLIST (MANDATORY)

1. Environment

- OS: Windows 10/11 (PowerShell recommended)
- Python: 3.10+ with project virtual environment available
- Node.js: 18+ and npm installed

2. Project Setup

- Repository root: d:\JudgeAI
- Backend dependencies installed in .venv
- Frontend dependencies installed in frontend/node_modules

3. Configuration

- Ensure .env exists in repository root with:
SUPABASE_URL
SUPABASE_KEY
GROQ_API_KEY
SUPABASE_STORAGE_BUCKET (optional; defaults if app handles it)

4. Database Migration

- Run SQL migration in Supabase SQL editor:
backend/db_migrations_upgrade_202602.sql
- Verify columns exist:
extracted_actions.action_plan
extracted_actions.action_plan_reasoning
cases.layout_blocks
cases.embedding

5. Clean Start (optional but recommended for full-cycle testing)

- Run:
backend/scripts/reset_all_data.sql
- Also clear Supabase Storage objects manually if you need a completely cold dataset.

SECTION 2 ? START THE APPLICATION

Terminal A (Backend):

```
Set-Location d:\JudgeAI  
.venv\Scripts\uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000
```

Terminal B (Frontend):

```
Set-Location d:\JudgeAI\frontend  
npm run dev
```

Open:

API Docs: <http://127.0.0.1:8000/docs>
Frontend: <http://localhost:5173>

Expected health:

GET / returns service/version response.

=====

SECTION 3 ? END-TO-END TEST FLOW (STEP BY STEP)

=====

STEP 1 ? Upload a PDF

- a) Open Home page
- b) Upload one judgment PDF
- c) Confirm upload success and generated case metadata

Expected:

- Record inserted in cases table
- Public/storage URL present

STEP 2 ? Run extraction pipeline

- a) Trigger extraction (single upload flow or /api/extract-actions)

Expected:

- extracted_data returned
- action_plan returned
- action_plan_reasoning returned
- extracted_actions row inserted with status pending

STEP 3 ? Verify base extracted fields

Check:

- case_number
- judgment_date
- department
- deadline
- directive
- source_sentence
- confidence_score (raw row-level model output/fused persistence)

STEP 4 ? Verify government action plan

Check in action_plan:

- department
- action_type
- priority_level
- appeal_recommended
- compliance_deadline
- appeal_deadline
- responsible_officer_role
- confidence_score (fused)

STEP 5 ? Verify verification workflow

- a) Open Verification Queue
- b) Approve one case
- c) Edit one case field
- d) Reject one case with reason

Expected:

- status transitions reflected
- audit logs created for each operation

STEP 6 ? Verify PDF highlight overlays (Case Details)

Open Case Details and verify color highlights:

- Directive highlighted (yellow, ?)
- Deadline highlighted (blue, ?)
- Party names/labels highlighted (green, ?)

STEP 7 ? Verify explainability panel visible (AI transparency)

On Case Details page, verify a visible "AI explainability" section showing:

- Why appeal recommended
- Why department selected

Data source:

- action_plan_reasoning

Notes:

- If missing on legacy rows, fallback guidance text should still be visible.

STEP 8 ? Verify confidence fusion is shown correctly

On Case Details radial gauge:

- Subtitle must read exactly: "Final Confidence Score"
- Score must represent weighted fused confidence, not LLM-only confidence.

Fusion includes:

- LLM extraction confidence
- timeline parser confidence
- department classifier confidence
- appeal recommender confidence

Also verify fusion transparency table:

- raw signal
- effective clamped signal
- subsystem weight
- weighted term contribution
- imputed flag for missing subsystem values

STEP 9 ? Verify admin analytics

Open Admin Dashboard and validate:

- System overview cards
- Government intelligence widgets
- Status charts and trends
- Confidence fusion analytics block with exact aggregated stats:
count, mean, min, max, pstdev
- CSV export includes fusion analytics + per-action fusion snapshots

STEP 10 ? Verify semantic search

Use /api/search-semantic with a legal/governance query.

Expected:

- Ranked results with similarity scores (if pgvector function configured)
- Graceful fallback behavior if RPC is unavailable

=====

SECTION 4 ? DETAILED FEATURE CHECKLIST

=====

A. Ingestion and storage

- Single PDF upload
- Batch upload with job status polling
- Case metadata persistence

B. Extraction and intelligence

- LLM structured extraction
- Timeline parser deadline inference
- Appeal recommendation engine
- Department classifier (embedding + heuristic selection)
- Confidence fusion (multi-signal weighted score)
- Explainability payload generation

C. UI and workflow

- Officer dashboard metrics
- Verification queue actions (approve/edit/reject)
- Case details with PDF visual highlights
- Explainability panel
- Final confidence radial gauge and fusion transparency
- Admin dashboard analytics + CSV export

D. Platform capabilities

- Notification scheduler (deadline alerts)
- Semantic search with embeddings
- Audit logging for traceability

SECTION 5 ? TECH STACK

Frontend

- React (Vite)
- Recharts (analytics charts)
- react-pdf-highlighter (PDF highlights)
- lucide-react (icons)

Backend

- FastAPI (REST APIs)
- Python services architecture (modular engines)
- APScheduler (scheduled notifications)
- PyMuPDF / fitz (PDF parsing + layout blocks)

Database and infrastructure

- Supabase (PostgreSQL + Auth + Storage)
- pgvector extension for semantic search
- JSONB action payloads for explainability/fusion

SECTION 6 ? AI/ML MODELS AND INTELLIGENCE COMPONENTS

1) LLM extraction model

- Groq-hosted LLaMA 3.x family
- Purpose: extract structured fields from judgment text

2) Embedding model

- sentence-transformers/all-MiniLM-L6-v2 (384 dimensions)
- Purpose:
 - a) semantic search embeddings for cases
 - b) department profile similarity scoring

3) Rule/NLP components

- Timeline parser (date phrase detection + offset handling)
- Appeal recommender heuristic classifier
- Confidence fusion weighted combiner with neutral imputation policy

SECTION 7 ? TEST DATA RECOMMENDATIONS

Use these data variants during QA:

- Compliance-focused judgments with explicit deadline
- Appeal-prone judgments with limitation clues
- Ambiguous department context
- Missing/weak date clues (to test parser fallback and imputation)
- Large PDFs and low-quality OCR text cases

Suggested in-repo fixture:

backend/fixtures/sample_judgment_rte_fixture.pdf

SECTION 8 ? FUTURE SCOPE AND BETTER MODIFICATIONS

1. Model and extraction quality

- Add domain fine-tuned legal extraction model benchmarking
- Introduce confidence calibration curves by subsystem
- Add multilingual extraction support (regional legal docs)

2. Explainability and governance

- Persist per-field provenance references (exact sentence/paragraph ids)
- Add downloadable explainability report per case
- Add policy-rule trace output for compliance audits

3. Analytics and monitoring

- Time-series drift monitoring for subsystem confidence
- Department-wise error heatmaps and false-positive tracking
- SLA dashboard for extraction latency and review turnaround

4. Workflow and product enhancements

- Multi-level reviewer assignment and escalation matrix
- Bulk verification actions with safeguards
- Stronger role-based access controls and row-level audit hardening

5. Search and retrieval

- Hybrid retrieval (keyword + vector + metadata filters)
- Better chunking and citation snippets in search results

6. Reliability and DevOps

- CI test suites (API + UI + regression fixtures)
- Auto backfill jobs for legacy rows missing action_plan_reasoning
- Structured observability (trace IDs, metrics, error budgets)

SECTION 9 ? FINAL ACCEPTANCE CRITERIA

Release can be marked QA-pass when:

- All end-to-end steps pass without blocking errors
- Explainability panel is visible and populated/fallback-safe
- Final Confidence Score reflects fusion, not single-model score
- Admin fusion analytics are numerically consistent with stored data
- Verification and audit logs are complete and reproducible

Document generated for JudgeAI testing and evaluation.